



Université Alioune Diop

L'Excellence est ma constance, l'Éthique ma vertu !

Automatisation et Orchestration dans le Cloud

Chapitre 1:

Architecture Cloud-Native et Etape/Outils d'automatisation

Dr. Cheikh Saliou Mbacke Babou

Objectif du Chapitre

À la fin de ce chapitre, vous devriez être en mesure de :

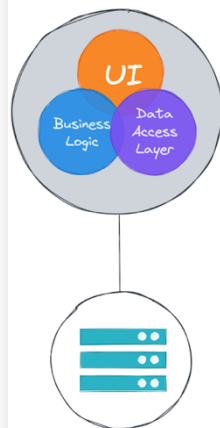
- Discuter des caractéristiques de l'architecture Cloud Native
- Expliquer les avantages de la mise à l'échelle automatique et des modèles sans serveur
- Comprendre l'importance des normes ouvertes et des projets communautaires dans les environnements cloud natifs.
- Expliquer les outils d'automatisation qui accélèrent le développement et le déploiement du code.
- Expliquer les bénéfices du stockage de l'infrastructure en tant que code.
- Expliquer comment les outils d'automatisation permettent de tester les déploiements d'applications.



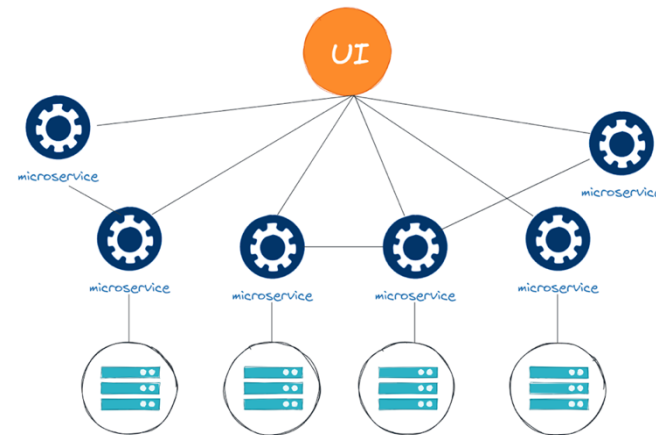
1.1. Principes de base de l'architecture cloud native

Les principes de base et caractéristiques de l'architecture Cloud Native

- **Concept et définition du terme cloud Native** (selon CNCF)
- *"Les technologies cloud natives permettent aux entreprises de créer et d'exécuter des applications évolutives dans des environnements modernes et dynamiques tels que les clouds publics, privés et hybrides."*
- **Applications traditionnelles ou Monolithiques**
- **Concept de micro services**
- **Haut niveau d'automatisation**
- **Autoguérison**
- **Evolutif**
- **Coût**
- **Facilité à entretenir**
- **Sécurisé par défaut**



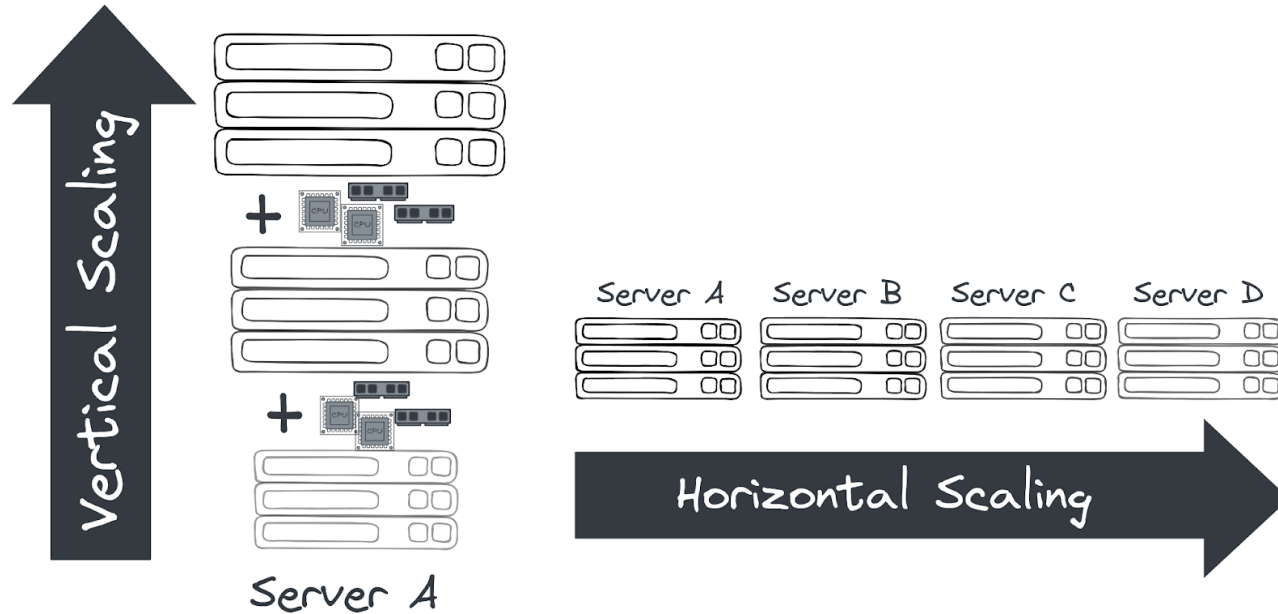
Monolithic Architecture



Microservice Architecture

Mise a l'échelle automatique (Autoscaling)

- Mise a l' échelle horizontal
- Mise a l' échelle vertical



Mise à l'échelle horizontale ou verticale

La notion de serverless (Sans serveur) et Normes ouvertes

Notion de serverless (Sans serveur)

- Serveless Egale sans serveur ?
- Function as a Service (FaaS)
- Le projet CloudEvents

Normes ouvertes

- Pourquoi?
- Open Container Initiative (OCI)
 - **image-Spec.**
 - **runtime-Spec**
 - **Distribution-Spec**
 - **OCI Specification**
 - **Container Network Interface (CNI)**
 - **Container Runtime Interface (CRI)**
 - **Container Storage Interface (CSI)**
 - **Service Mesh Interface (SMI)**

Rôles Natifs du Cloud et Ingénierie de la fiabilité des sites

Rôles natifs du Cloud

- **Architecte Cloud**
- **Ingénieur DevOps**
- **Ingénieur Sécurité**
- **Ingénieur SecDevOps**
- **Ingénieur de données**
- **Développeur Full-Stack**
- **Ingénieur en fiabilité de site (SRE):**
 - **Objectifs de niveau de service (SLO)**
 - **Indicateurs de niveau de service (SLI)**
 - **Accords de niveau de service (SLA)**

Niveau de Maturité des Projets

- **Phase Sandbox**
- **Phase d' incubation**
- **Phase de graduation**



1.2. Les Scripts d'automatisation de base

Outils de base pour l'automatisation des scripts

Les shells sont omniprésents, donc le script shell est historiquement le socle de l'automatisation.

Bash

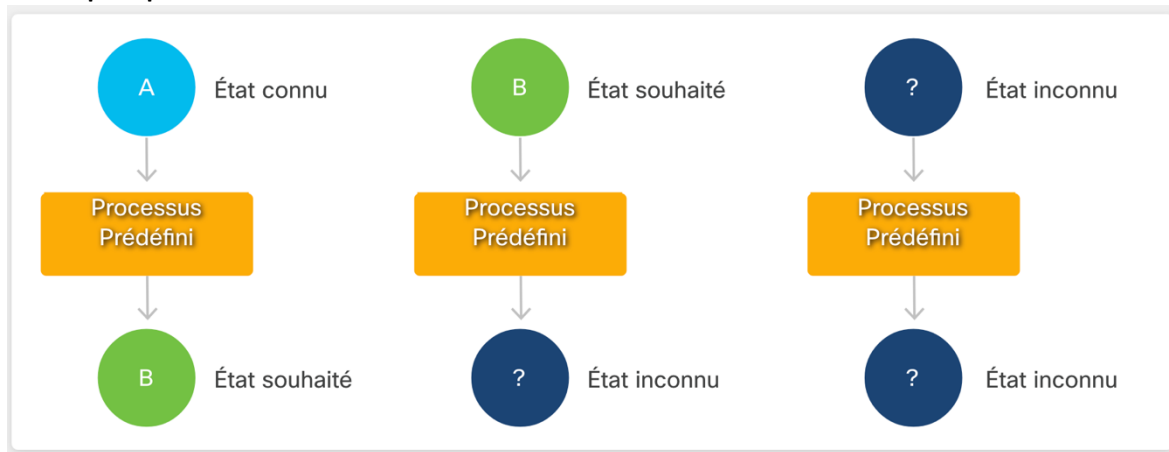
- Le Bash est un shell Unix et est un shell par défaut sur la plupart des distributions Linux et sur macOS. L'utilisation de commandes dans un script Bash est similaire à celle de les utiliser directement à partir de la ligne de commande.
- Bash peut être utilisé pour l'accès par script à l'interface de ligne de commande AWS, vous pouvez utiliser les fonctionnalités intégrées et les bibliothèques de langages plus sophistiqués pour analyser des jeux de données complexes retournés (tels que JSON), gérer de nombreuses opérations parallèles, traiter des erreurs, gérer les réponses asynchrones aux commandes, etc.

Les langages de programmation au-delà de Bash

- Les langages sophistiqués s'améliorent sur Bash lorsque la complexité et les exigences d'échelle augmentent. Ils sont utiles lors de la création et de l'exploitation d'infrastructures virtualisées dans des environnements cloud, à l'aide de kits SDK.

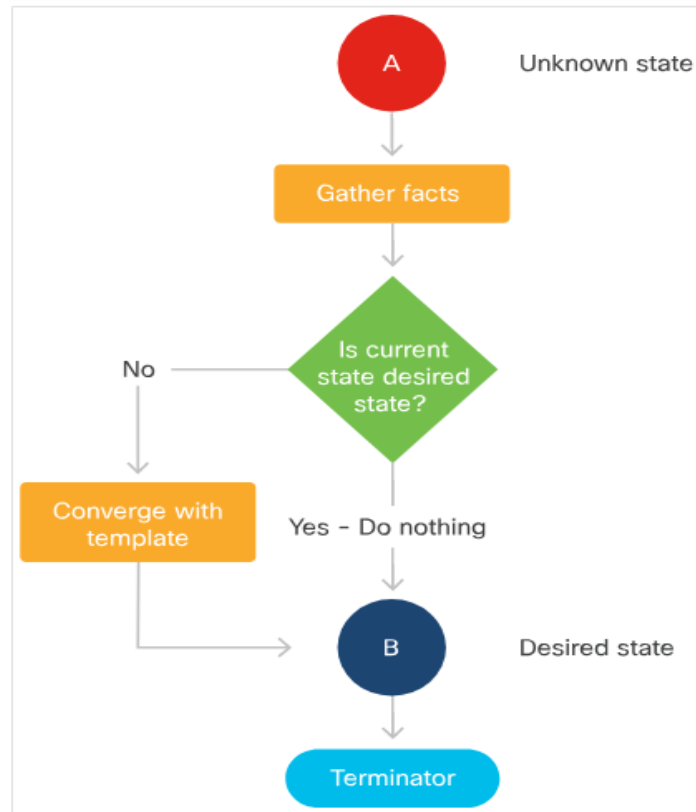
Automatisation des procédures

- Une procédure impérative est une séquence ordonnée de commandes visant à atteindre un objectif. La séquence peut inclure le contrôle du débit, les conditions, la structure fonctionnelle, les classes et plus encore.
- Pour assurer l'efficacité et la réutilisation des scripts, on peut :
 - Normaliser l'ordre et la présentation des paramètres, des indicateurs et des erreurs.
 - Créer une hiérarchie de code qui divise les tâches de manière logique et efficace.
 - Créer des scripts de haut niveau pour des déploiements entiers et des scripts de niveau inférieur pour les phases de déploiement.
 - Séparer les données spécifiques au déploiement du code, ce qui rend le code aussi générique et réutilisable que possible.



Automatisation des procédures (suite)

- Des scripts procéduraux et des outils de configuration déclarative soigneusement rédigés examinent les cibles avant d'effectuer des tâches sur elles et n'exécutent que les tâches nécessaires pour atteindre l'état souhaité.
- Cette qualité de logiciel est appelée idempotence.
- Les principes de base de l'idempotence sont :
 - Assurez-vous que la modification que vous souhaitez apporter n'a pas déjà été effectuée
 - Accéder à un état dont le fonctionnement a été vérifié, si possible, avant d'apporter des modifications
 - Test d'idempotence
 - Tous les composants d'une procédure doivent être idempotent

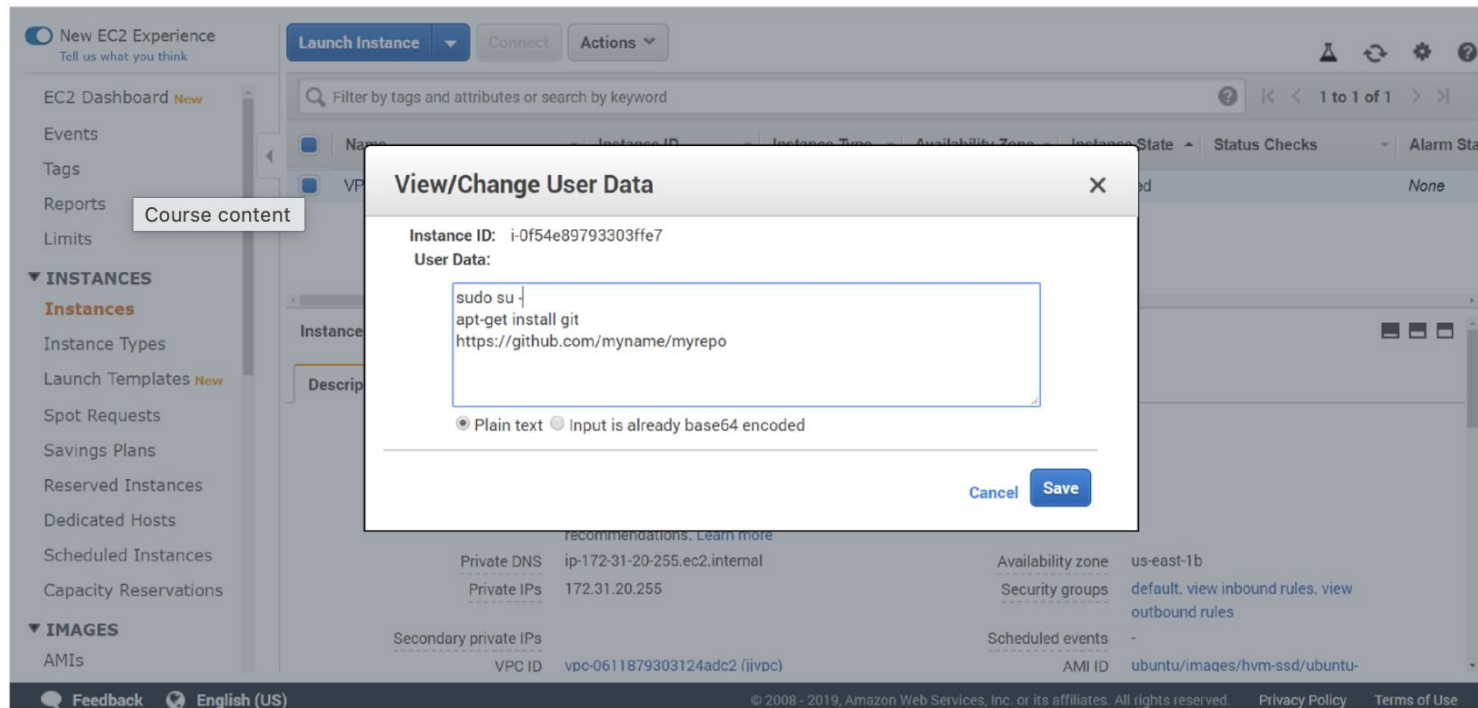


Exécution de scripts localement et à distance

Pour configurer des systèmes distants, l'utilisateur doit accéder et exécuter des scripts sur eux. Il y a plusieurs façons de le faire :

- Stockez les scripts localement, transmettez-les aux machines cibles avec un utilitaire shell comme `scp`, puis connectez-vous à la machine distante en utilisant `ssh` et exécutez-les.
- Canailisez des scripts vers une machine distante en utilisant `cat | ssh` et exécutez-les en séquence avec d'autres commandes, capturant et renvoyant les résultats au terminal, le tout en une seule commande.
- Installez un client de transfert de fichiers sécurisé à usage général comme **SFTP**, puis utilisez cet utilitaire pour vous connecter à la machine distante, transférer, définir les autorisations appropriées, puis exécuter le fichier de script.
- Stockez des scripts sur un serveur web, connectez-vous à la machine distante et récupérez-les avec `wget`, `curl` ou d'autres utilitaires, ou stockez les scripts dans un dépôt Git.
- Installez une solution complète d'opérations à distance comme VNC ou NoMachine localement, installez son serveur sur la cible, transmettre/copier, puis exécutez des scripts.
- Si les machines cibles sont provisionnées sur une infrastructure cloud, il existe généralement un moyen d'injecter un script de configuration via la même commande CLI ou l'action WebUI qui manifeste la plate-forme.

Exécution de scripts localement et à distance



Automatisation du cloud

- L'automatisation du cloud permet à l'utilisateur de provisionner des hôtes virtualisés, de configurer des réseaux virtuels et d'autres services de connectivité, de réquisition, puis de déployer des applications sur cette infrastructure.
- Les fournisseurs de cloud et les communautés open source fournissent souvent des sous-systèmes spécialisés pour les outils de déploiement populaires, qui extraient un inventaire complet des ressources d'une infrastructure cloud et le tiennent à jour en temps réel pendant que l'automatisation apporte des modifications.
- L'utilisateur peut également gérer des ressources cloud à l'aide de scripts écrits en Bash, Python ou dans d'autres langues.

Cloud CLI et SDK

Les IaaS et d'autres types de cloud d'infrastructure fournissent également des CLI et des SDK qui permettent une connexion facile à leurs interfaces sous-jacentes, généralement basées sur REST.

Cisco UCS - un cloud de métal nu	• API RESTful Cisco Intersight • Gamme de kits SDK pour l'API Intersight RESTful, y compris ceux pour Python et Microsoft PowerShell • Gamme de modules Ansible
VMware	• Datacenter CLI • vSphere CLI pour Linux et Windows • PowerCLI pour Windows PowerShell • Hôte de kits SDK pour les langages populaires, destinés à vSphere Automation, vCloud Suite et d'autres produits
OpenStack	• Client OpenStack (OSC) • API OpenStack de calcul, d'identité, d'image, de stockage d'objets et de stockage en bloc • Kit de développement logiciel OpenStack Python • Boîte à outils OpenStack



1.3. Les outils d'automatisation

Que font les outils d'automatisation pour nous ?

Que font les outils d'automatisation pour nous ?

Les outils d'automatisation offrent des fonctionnalités puissantes par rapport aux stratégies d'automatisation ad hoc utilisant **BASH**, **Python** ou d'autres langages de programmation. Ces outils permettent aux développeurs de :

- Simplifier et uniformiser
- Accélérer le développement grâce à des fonctionnalités prêtes à
- Faciliter la réutilisation, séparer les préoccupations et promouvoir la sécurité
- Effectuer la découverte et gérer l'inventaire
- Balance de poignée (Handle scale)
- Mobiliser la communauté

Concepts critiques

Idempotence : un aperçu

- Un logiciel Idempotent produit le même résultat souhaitable chaque fois qu'il est exécuté.
- Dans un logiciel de déploiement, Idempotence permet la convergence et la composabilité et permet de :
 - Facilite de rassemblement des composants
 - Collections complètes de construction/déploiement

Procédure par opposition à déclaration

Concepts critiques

```
- name: Install Apache webserver
  apt:
    name: apache2
    state: present
    update-cache: yes
```

Concepts critiques (suite)

Provisionnement vs. configuration vs. déploiement vs. orchestration

Provisionnement	Configuration	Déploiement	Orchestration
Il s'agit d'obtenir une infrastructure de calcul, de stockage et de réseau (réelle ou virtuelle), d'activer les communications, de la mettre en service et de la rendre prête à l'emploi par les opérateurs et les développeurs.	Cela signifie installer des applications et des services de base et effectuer les opérations, les tâches et les tests nécessaires pour préparer une plate-forme de bas niveau pour déployer des applications ou une plate-forme de niveau supérieur.	Cela implique la création, l'organisation, l'intégration et la préparation d'applications multi-composants ou de plates-formes de niveau supérieur, souvent sur plusieurs nœuds.	Cela peut se référer à plusieurs choses : • Automatisation construite par l'utilisateur ou inhérente à la plate-forme visant à gérer les cycles de vie des charges de travail et à réagir dynamiquement aux conditions changeantes. • Des processus ou des workflows qui relient les tâches d'automatisation pour offrir des avantages commerciaux, tels que le libre-service (self-service).

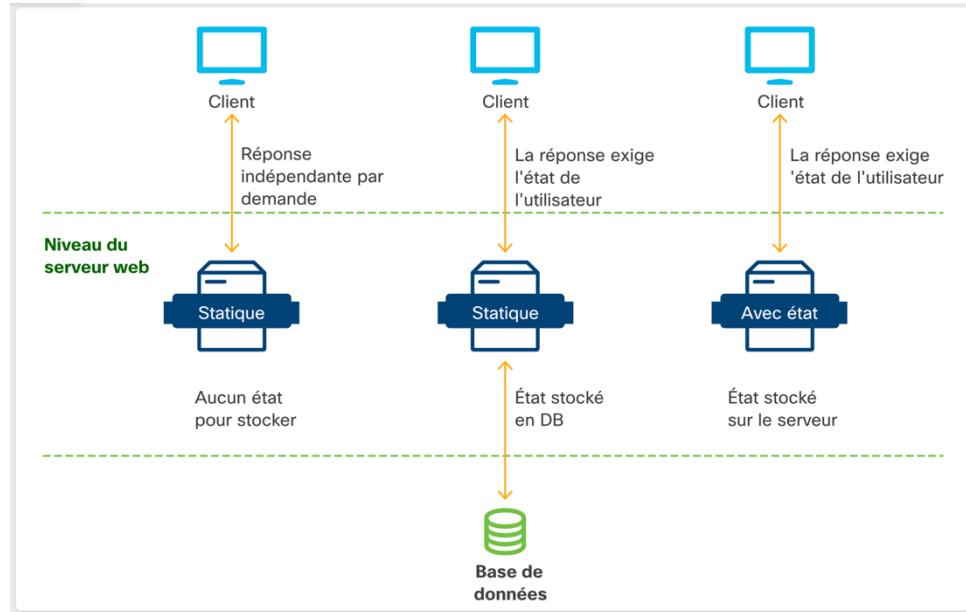
Concepts critiques (suite)

Sans état (Statelessness)

L'automatisation fonctionne mieux lorsque les applications peuvent être rendues sans état. Cela signifie que leur redéploiement en place ne détruit ni ne perd aucune trace des données dont les utilisateurs ou les opérateurs ont besoin.

Les deux états d'une demande sont les suivants :

- **Pas apatriide (Not Stateless)** : application qui enregistre des informations importantes dans des fichiers ou dans une base de données sur le fichier local.
- **Apatriides (Stateless)** : application qui conserve son état dans une base de données distincte ou qui fournit un service qui ne nécessite aucune mémoire d'état entre les invocations.



Outils d'automatisation populaires

- Le premier outil d'automatisation moderne a été **Puppet** qui a été introduit en 2005 comme open source, puis commercialisé sous le nom de **Puppet Enterprise** par Puppet Labs en 2011.
- Les outils d'automatisation les plus populaires sont Ansible, Puppet, Chef. Ils partagent les caractéristiques suivantes :
 - Relativement facile à apprendre
 - Disponible en version open source
 - Les plugins et les adaptateurs leur permettent de contrôler directement ou indirectement de nombreux types de ressources
- De nombreuses autres solutions existent également. Les fournisseurs de cloud privés et publics approuvent souvent leurs propres outils pour une utilisation sur leurs plateformes, tels que le projet HEAT d'OpenStack, AWS CloudFormation, SaltStack et Terraform.

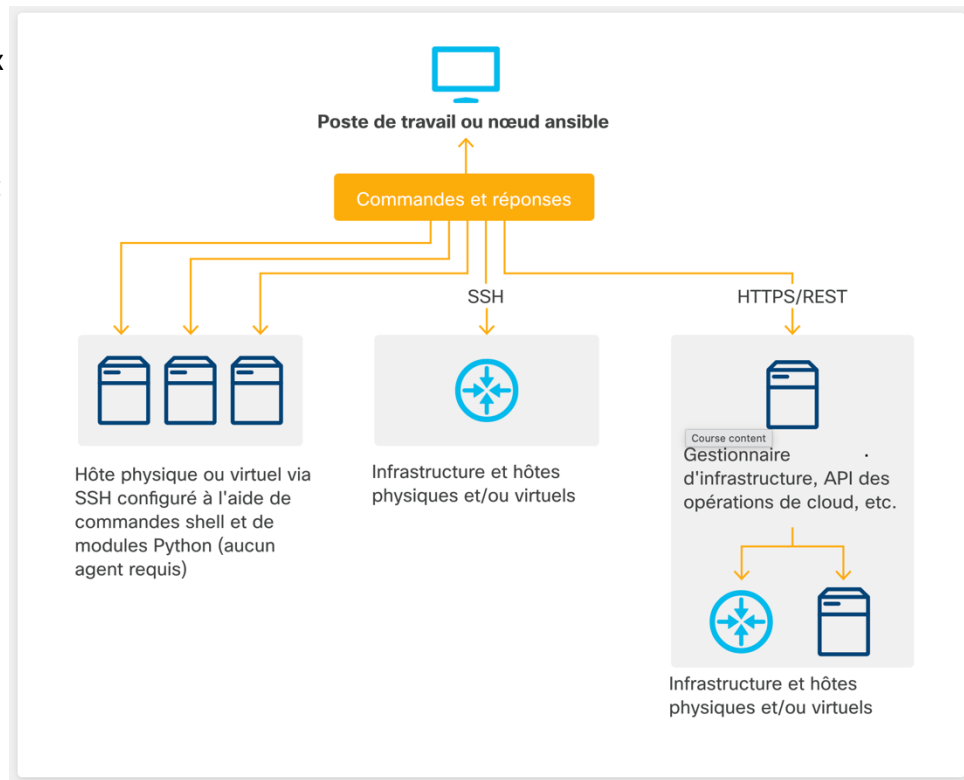
Ansible

- Ansible est disponible en open source, et dans une version avec des fonctionnalités supplémentaires, de IBM/Red Hat, appelée Ansible Tower.
- Ansible est largement géré à partir de la ligne de commande Bash, avec un code d'automatisation développé et maintenu à l'aide de n'importe quel éditeur de texte standard.

Ansible (suite)

Architecture Ansible : Simple et Léger

- Le nœud de contrôle s'exécute sur n'importe quelle machine Linux exécutant Python 2 ou 3. Toutes les mises à jour du système sont effectuées sur le nœud de contrôle.
- Le nœud de contrôle se connecte aux ressources gérées via SSH et permet à Ansible de :
 - Exécutez des commandes shell sur un serveur distant, ou effectuez des transactions avec un routeur distant, ou une autre entité réseau, via son interface REST.
 - Injectez des scripts Python dans les cibles et supprimez-les après leur exécution.
 - Installez Python sur les machines cibles si nécessaire.
- Les plugins permettent à Ansible de collecter des faits et d'effectuer des opérations sur une infrastructure qui ne peut pas exécuter Python localement.



Ansible (suite)

Installation d'Ansible

- L'application de nœud de contrôle Ansible est installée sur une machine Linux à partir de son dépôt de paquets public. Pour installer Ansible sur un poste de travail, reportez-vous à la documentation d'installation appropriée au périphérique.

Structure de code ansible

- Dans la structure de code Ansible, le travail est séparé en fichiers YAML (`.yaml`) qui contiennent une séquence de tâches, exécutées dans l'ordre de haut en bas. Ansible a des centaines de modules Python pré-construits qui enveloppent des fonctions au niveau du système d'exploitation et des méta-fonctions.

Playbooks et rôles

- Un playbook Ansible peut être écrit comme un document monolithique avec une série de tâches nommées et modulaires.
- Les développeurs construisent un modèle d'une tâche DevOps complexe à partir de séquences de tâches de playbook de bas niveau appelées rôles, puis référencez-les dans des playbooks de niveau supérieur, ajoutant parfois des tâches supplémentaires au niveau du playbook.
- Cette séparation des préoccupations assure la clarté, la réutilisation et la partageabilité des rôles.

Ansible (suite)

- Exemple de Playbook

```
---
- hosts: all
  become: true
  gather_facts: False
  tasks:
    - name: install python 2
      raw: test -e /usr/bin/python || (apt -y update && apt install -y python-minimal)
- hosts: all
  become: true
  roles:
    - configure-nodes
- hosts: k8smaster
  become: true
  roles:
    - create-k8s-master
    - install-weave
    - wrapup
- hosts:
  - k8sworker1
  - k8sworker2
  become: true
  roles:
    - join-workers
...
```

Ansible (suite)

Organisation du projet Ansible:

- Fichiers d'inventaire
- Fichiers de variable
- Fichiers de bibliothèque et utilitaires
- Fichiers principaux de playbook
- Dossiers et fichiers de rôles

Ansible à l'échelle

- Il existe des défis de taille pour les grandes entreprises, tels que la gestion et le contrôle de l'accès à de nombreux nœuds Ansible de manière flexible et sécurisée. Cela inclut également la mise en place de contrôleurs à distance en toute transparence et en toute sécurité sous le contrôle de l'automatisation centralisée de l'entreprise.
- Pour cela, il existe deux solutions de plan de contrôle : Red Hat Ansible Tower et AWX projet.
- Les implémentations d'Ansible à plus grande échelle bénéficient également d'Ansible Vault, une fonctionnalité intégrée qui permet le chiffrement des mots de passe et d'autres informations sensibles.

Ressources de Cisco Ansible

Exemple Ansible

- Ansible utilise normalement `ssh` pour se connecter à des hôtes distants et exécuter des commandes.
- Voyons comment créer un site Web simple sur un hôte distant.

Connaissances préalables requises

- Un hôte cible exécutant un système d'exploitation compatible (tel que le serveur Ubuntu 18.04)
- SSH et authentification par clé configurée sur cet hôte
- Ansible installé sur votre poste de travail local

Exemple Ansible (suite)

Construction d'une arborescence de fichiers de projet Ansible

- Aux fins de cet exercice, le nom d'hôte de la machine cible (résolution DNS) est la **Target**.
- Avec votre machine cible accessible en SSH-accessible, commencez à construire une structure de dossier de base pour le projet Ansible.
- Au niveau supérieur de votre dossier de projet, vous avez besoin des éléments suivants :
 - Fichier d'inventaire contenant des informations sur la ou les machines sur lesquelles vous souhaitez déployer.
 - Un fichier **site.yml** de haut niveau, contenant le niveau d'instructions le plus abstrait.
 - Structure de dossier de rôle pour contenir votre rôle de **serveur Web**.

```
mkdir myproject  
cd myproject
```

```
touch inventory  
touch site.yml  
mkdir roles  
cd roles  
ansible-galaxy init webserver
```

Exemple Ansible (suite)

Création de votre fichier d'inventaire

- Votre fichier d'inventaire pour ce projet peut être très simple. Faites-en le nom d'hôte résoluble par le DNS de votre machine cible :

```
[webservers]  
target # can also be IP address
```

- Vous définissez un groupe appelé **serveurs Web** et y mettez le nom d'hôte (ou IP) de votre machine cible.
- Vous pouvez ajouter de nouveaux noms d'hôte/IP à ce bloc de groupe ou ajouter des blocs de groupe supplémentaires pour affecter des hôtes à des déploiements plus complexes.

```
[webservers]  
target1  
target2  
target3  
[dbservers]  
target4  
target5  
target6
```

Exemple Ansible (suite)

Création de votre fichier de playbook de niveau supérieur

- Un playbook de niveau supérieur décrit généralement l'ordre, les autorisations et d'autres détails sous lesquels les actions de configuration de niveau inférieur, définies dans les rôles, sont appliquées.
- Dans cet exemple, le fichier `site.yml` identifie les hôtes sur lesquels vous souhaitez effectuer une opération et les rôles que vous souhaitez appliquer à ces hôtes.
- La ligne `become: true` indique à Ansible que vous voulez jouer les rôles en tant que root, via sudo.

```
---  
- hosts: webservers  
  become: true  
  roles:  
    - webservers
```

Création de votre rôle de serveurs Web

- L'étape suivante consiste à créer le rôle qui installe et configure votre serveur Web.
- Vous avez déjà créé la structure de dossier pour le rôle en utilisant `ansible-galaxy`.
- Le code du rôle est contenu dans un fichier appelé `main.yml` dans le répertoire `/tasks` du rôle.

Exemple Ansible (suite)

- Vous pouvez éditer le fichier `rôles/webserver/tasks/main.yml` directement, comme illustré ici.
- Le rôle comporte deux tâches :
 - Déployez Apache2.
 - Copiez un nouveau fichier `index.html` dans la racine HTML Apache2, en remplaçant la page `index.html` par défaut.
- Dans `apt`, vous nommez le paquet, son état requis et demandez au module apt de mettre à jour son cache.
- Dans la seconde strophe, la routine de copie d'Ansible déplace un fichier de votre système local vers un répertoire sur la cible et modifie également son propriétaire et ses autorisations.

```
---  
- name: Perform updates and install apache2  
  apt:  
    name: apache2  
    state: present  
    update_cache: yes  
- name: Insert new homepage index.html  
  copy:  
    src: index.html  
    dest: /var/www/html  
    owner: myname  
    mode: '0444'
```

Exemple Ansible (suite)

Création de votre fichier index.html

- Bien sûr, vous devrez également créer un nouveau fichier `index.html`.
- La commande Ansible `copy` suppose que ces fichiers seront stockés dans le répertoire `/files` du rôle qui les appelle, sauf indication contraire.
- Accédez à ce répertoire et créez le fichier `index.html`, en enregistrant vos modifications par la suite.

```
<html>
<head>
<title>My Website</title>
</head>
<body>
<h1>Hello!</h1>
</body>
</html>
```

Exemple Ansible (suite)

Exécution de votre déploiement

- Maintenant, vous êtes prêt à exécuter votre déploiement. Dans le répertoire de niveau supérieur de votre projet, vous pouvez le faire avec l'instruction :

```
ansible-playbook -i inventory -u myname -K site.yml
```

-i nomme votre fichier d'inventaire.

-u nomme votre utilisateur sudo.

-K demande à Ansible de nous demander votre mot de passe sudo, au moment où il commence l'exécution.

site.yml est le fichier qui régit votre déploiement.

Exemple Ansible (suite)

- Si tout va bien, Ansible devrait nous demander votre mot de passe DEVEINE (mot de passe sudo), puis retourner les résultats similaires au suivant :

```
BECOME password:
PLAY [webservers] *****
TASK [Gathering Facts] *****
ok: [192.168.1.33]
TASK [webservers : Perform updates and install apache2] *****
changed: [192.168.1.33]
TASK [webservers : Insert new homepage index.html] *****
changed: [192.168.1.33]
PLAY RECAP *****
192.168.1.33      : ok=3    changed=2    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
```

- Et maintenant, si vous visitez l'adresse IP de votre machine cible dans un navigateur, vous devriez voir votre nouvelle page d'accueil.

Exemple Ansible (suite)

Procédure pas à pas Ansible CI/CD

Parcourons l'exemple comme s'ils faisaient partie d'un pipeline CI/CD.

- Développeur collaborant avec vous sur GitHub commet un changement sur le site Web tel que dans le fichier `index.html`.
- Ensuite, les tests dans le référentiel exécutent des contrôles de syntaxe ainsi que des règles de révision de code pour chaque requête d'extraction.
- Ensuite, le système CI/CD prépare un environnement et exécute des tests prédéfinis pour n'importe quel playbook Ansible. Il doit indiquer la version attendue à chaque fois et l'installer. Voici un exemple de pipeline :

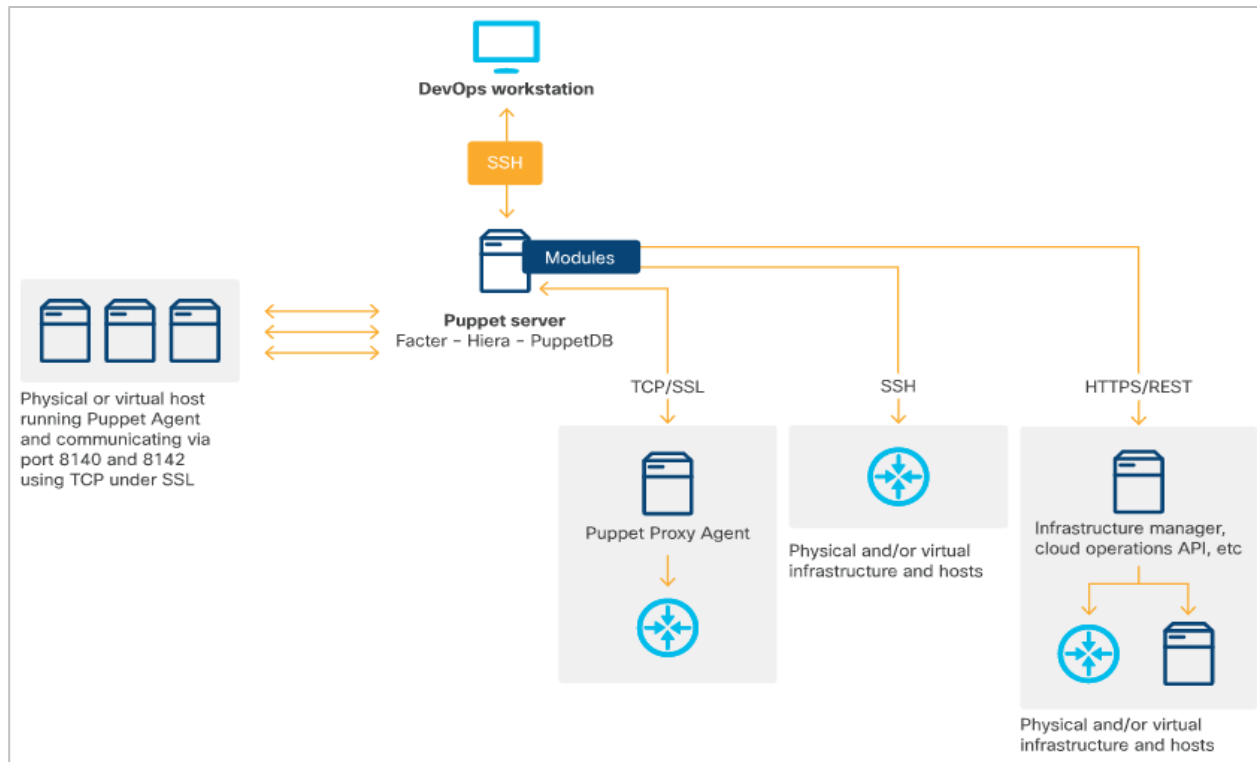
```
pip install ansible==2.9.2
ansible-version
ansible-playbook main.yml --syntax-check
ansible-playbook -i staging-servers.cfg main.yml -check
ansible-playbook -i staging-servers.cfg main.yml -vvvv
```

Puppet

Fondée comme open source en 2005 et commercialisée sous le nom de Puppet Enterprise par Puppet Labs en 2011.

Architecture components

- Serveur désigné pour héberger les principaux composants de l'application.
- Un client sécurisé, également connu sous le nom d'agent Puppet.
- Modules permettant d'activer les connexions pour les API cloud et le matériel qui ne peuvent pas exécuter un agent.
- Dans les implémentations évolutives, un agent proxy pour décharger le travail de connexion directe aux CLI de périphérique et d'échange d'informations.



Puppet (suite)

Installation de Puppet

- Puppet Server nécessite un matériel puissant (ou une grande machine virtuelle) et un client Network Time Protocol pour être installé, configuré et testé.
- Les agents auront besoin du fichier `puppet.conf` configuré pour communiquer avec le serveur Puppet.
- Une fois le service client démarré, le certificat sera signé par le serveur. Le serveur sera désormais en mesure de rassembler les faits du client et de mettre à jour l'état du client avec toute modification de configuration.

Structure du code de marionnette

- Puppet stocke les composants d'un projet ou d'une configuration discrète dans une arborescence de répertoires (`/etc/puppetlabs/code/environments`).
- Les dossiers subsidiaires sont créés en fonction de la configuration dans `puppet.conf` ou par l'opérateur.
- Puppet est livré avec un ensemble de ressources de base intégrées. De nombreuses ressources supplémentaires pour effectuer toutes sortes d'opérations peuvent être téléchargées et installées à partir de Puppet Forge à l'aide de la commande du module Puppet.

Puppet (suite)

Puppet à l'échelle

- Puppet Server est un peu monolithique, et une installation monolithique est recommandée par les implémentateurs (open source).
- L'étape pour accueillir plus d'hôtes consiste à créer des « maîtres de compilation » supplémentaires, qui compilent des catalogues pour les agents clients et les placent derrière un équilibreur de charge pour distribuer le travail.
- Les clients de Puppet Enterprise peuvent accroître leur capacité en remplaçant PuppetDB par une base de données autonome et personnalisée appelée PE-PostgreSQL.

Ressources de Puppet Cisco

Cisco et la communauté Puppet maintiennent de vastes bibliothèques de modules pour automatiser le matériel de calcul et de réseau Cisco. Notamment :

- Modules Cisco IOS permettant la gestion de l'infrastructure IOS
- Modules Cisco UCS permettant le contrôle de UCS via UCS Manager

Exemple de Puppet

- Cet exercice explique comment installer Puppet, puis utiliser Puppet pour installer Apache 2 sur un appareil.
- Cela correspond approximativement au flux de travail normal pour les opérations Puppet dans un environnement client/serveur automatisé.
- Notez que les modules peuvent être complètement génériques et exempts d'informations spécifiques au site, puis être appelés séparément et réutilisables pour configurer un nombre quelconque d'hôtes ou de composants d'infrastructure.

Installation de Puppet Server

- Puppet Server nécessite un matériel puissant (ou une grande machine virtuelle) et un client Network Time Protocol pour être installé, configuré et testé.
- Les instructions pour l'installation du serveur sont disponibles dans la documentation de Puppet.

Exemple de Puppet (suite)

Installation du client Puppet

- Lorsque le serveur Puppet Server est en cours d'exécution, vous pouvez installer Puppet Agents sur un hôte. Par exemple, sur un système Linux de type Debian, vous pouvez installer Puppet Agent à l'aide d'une seule commande :

```
sudo apt-get install puppet-agent
```

Modification

- Une fois installé, Puppet Agent doit être configuré pour rechercher un serveur Puppet.
- Ajoutez les lignes au fichier `/etc/puppet/puppet.conf` qui indique au client, le nom d'hôte de votre serveur et le nom du certificat d'authentification que vous allez générer à l'étape suivante.

```
[main]
certname = puppetclient
server = puppetserver
environment = production
runinterval = 15m
```

Exemple de Puppet (suite)

- Démarrez le service de marionnettes sur le Client :

```
sudo /opt/puppetlabs/bin/puppet resource service puppet ensure=running enable=true
```

- Vous devriez obtenir une réponse similaire à la suivante :

```
Notice: /Service[puppet]/ensure: ensure changed 'stopped' to 'running'  
service { 'puppet':  
  ensure => 'running',  
  enable => 'true',  
}
```

Exemple de Puppet (suite)

Signature du certificat

- Les agents Puppet utilisent des certificats pour s'authentifier auprès du serveur avant de récupérer leurs configurations.
- Lorsque le service Client démarre pour la première fois, il envoie une demande à son serveur assigné pour que son certificat soit signé, ce qui permet la communication.
- Sur le serveur, émettez la commande `ca list` renvoie une liste de certificats en attente.

```
sudo /opt/puppetlabs/bin/puppetserver ca list
```

La réponse devrait être semblable à la suivante :

```
Requested Certificates:
```

```
puppetclient (SHA256)
```

```
44:9B:9C:02:2E:B5:80:87:17:90:7E:DC:1A:01:FD:35:C7:DB:43:B6:34:6F:1F:CC:DC:C2:E9:DD:72:61:E6:B2
```

Exemple de Puppet (suite)

- Vous pouvez ensuite signer le certificat, en activant la gestion du nœud distant :

```
sudo /opt/puppetlabs/bin/puppetserver ca sign --certname puppetclient
```

- La réponse :

```
Successfully signed certificate request for puppetclient
```

- Le serveur et le client sont désormais liés de manière sécurisée et capables de communiquer.
- Cela permettra au serveur de recueillir des faits à partir du client et vous permettra de créer sur le serveur des configurations obtenues par le client et utilisées pour converger son état (toutes les 15 minutes).

Exemple de Puppet (suite)

Créer une configuration

- Puppet vous permet de stocker les composants d'un projet ou d'une configuration discrète dans une arborescence de répertoires.

```
/etc/puppetlabs/code/environments
```

- Les dossiers subsidiaires sont créés en fonction de la configuration dans `puppet.conf` ou par l'opérateur.
- Dans cet exemple, ayant déclaré `environnement = production`, Puppet a déjà créé un répertoire pour ce site par défaut, contenant un sous-répertoire de `modules` dans lequel nous pouvons stocker des projets secondaires et des manifestes pour les choses que nous devons construire et configurer.

```
/etc/puppetlabs/code/environments/production/modules
```

Exemple de Puppet (suite)

- Vous allez maintenant installer Apache2 sur votre client géré. Les opérations de marionnettes sont généralement effectuées en tant que root, donc devenez root sur le serveur temporairement en entrant `sudo su -`.
- Accédez au dossier `/modules` dans l'environnement `/production` .

```
cd /etc/puppetlabs/code/environments/production/modules
```

- Créez une structure de dossier pour contenir le module `install apache` .

```
mkdir -p apache2/manifests  
cd apache2/manifests
```

- Dans le dossier manifests, créez un fichier appelé `init.pp`, qui est un nom de fichier réservé pour l'étape d'initialisation d'un module.

Exemple de Puppet (suite)

- La définition de classe ordonne les étapes que nous voulons effectuer :
 - **Étape 1.** Appelez la ressource de `paquet` pour installer le paquet nommé en `ensure => installed`.
 - **Étape 2.** Appelez la ressource de service à exécuter si son exigence est satisfaite. Demandez au serveur de s'assurer que le service est disponible, puis de le redémarrer automatiquement lorsque le serveur redémarre.

```
class apache2 {  
  package { 'apache2':  
    ensure => installed,  
  }  
  service { 'apache2':  
    ensure  => true,  
    enable  => true,  
    require => Package['apache2'],  
  }  
}
```

Exemple de Puppet (suite)

- Accédez au dossier des manifestes associé.

```
cd /etc/puppetlabs/code/environments/production/manifests
```

- Créez un fichier `site.pp` qui appelle le module et l'applique à la machine cible.

```
node 'puppetclient' {  
  include apache2  
}
```

Exemple de Puppet (suite)

Déploiement de la configuration

Vous disposez de deux options pour déployer la configuration terminée :

- Le redémarrage du serveur Puppet permet désormais de compiler les manifestes et de les mettre à la disposition de l'agent Puppet sur le périphérique nommé. L'agent va les récupérer et les appliquer, en installant Apache2 avec le prochain cycle de mise à jour :

```
systemctl restart puppetserver.service
```

- Pour le développement et le débogage, vous pouvez appeler Puppet Agent sur une machine cible. L'agent interroge immédiatement le serveur, télécharge son catalogue et l'applique. Les résultats seront similaires aux suivants :

```
sudo puppet agent -t
```

Exemple de Puppet (suite)

- L'agent interroge immédiatement le serveur, télécharge son catalogue et l'applique. The results will be similar to the following:

```
root@target:/etc/puppetlabs/code/environments/production/manifests# puppet agent -t
Info: Using configured environment 'production'
Info: Retrieving pluginfacts
Info: Retrieving plugin
Info: Retrieving locales
Info: Caching catalog for puppetagent
Info: Applying configuration version '1575907251'
Notice: /Stage[main]/Apache2/Package[apache2]/ensure: created
Notice: Applied catalog in 129.88 seconds
```

- Une fois que l'application a été déployée avec succès, entrez l'adresse IP de la machine cible dans votre navigateur. Cela devrait faire apparaître la page d'accueil par défaut d'Apache 2.

Chef

- Chef fournit un système complet pour traiter l'infrastructure en tant que code.
- Les produits Chef sont partiellement sous licence, mais sont gratuits pour un usage personnel (dans le cas de Chef Infra Server, pour moins de 25 nœuds gérés).
- Les produits et solutions de Chef permettent la création, le test, l'organisation, le stockage du référentiel et l'exécution sur des cibles distantes, soit à partir d'une station de travail Chef autonome, soit indirectement à partir d'un serveur Chef Infra central.

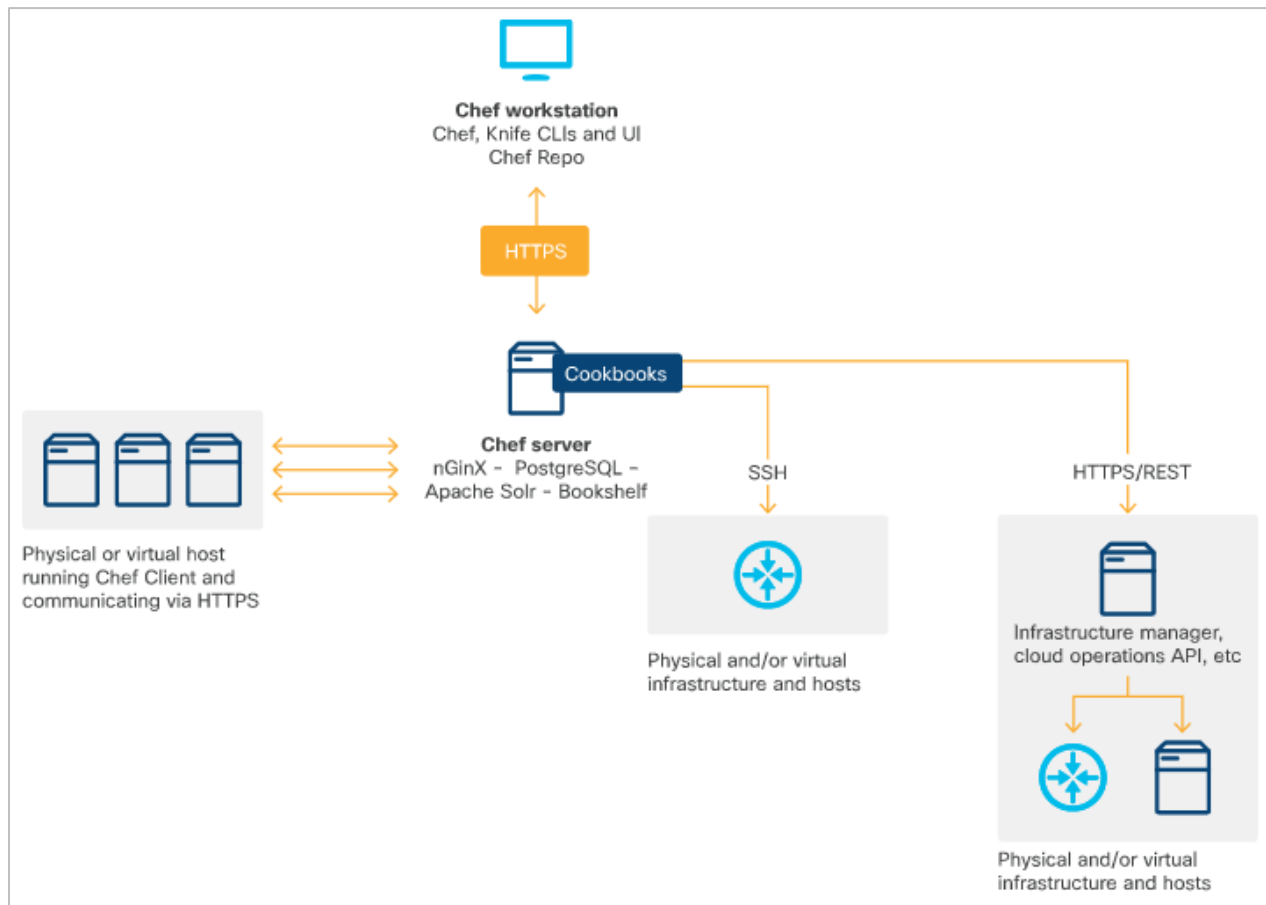
Outils d'automatisation

Chef (suite)

Chef Architecture - Composantes

- Chef Workstation
- Chef Infra Client (l'agent hôte)
- Chef Infra Server

La plupart des tâches de configuration peuvent également être effectuées directement entre Chef Workstation et les nœuds et périphériques gérés.



Chef (suite)

Les composants de Chef Workstation incluent des outils de ligne de commande, l'interaction avec les serveurs Chef Infra, Test Kitchen, ChefSpec et InSpec.

Installation de Chef Workstation

- Pour commencer à utiliser Chef, la première étape consiste à installer Chef Workstation, qui fournit un environnement opérationnel complet. Reportez-vous à la page de téléchargement de Chef Workstation pour plus d'informations.
- La station de travail est disponible pour Linux et Windows.

Faire fonctionner Chef à l'échelle

- Chef Infra Server peut être configuré pour une haute disponibilité en déployant ses services frontaux dans une gamme de proxys équilibrés de charge.
- Chef propose également une gamme de produits qui, ensemble, résolvent la plupart des problèmes rencontrés par les entreprises face à des infrastructures hybrides de plus en plus complexes et à grande échelle.

Chef (suite)

Ressources du Chef Cisco

- Cisco a développé des agents Chef Infra modifiés qui s'exécutent dans le shell invité des équipements de commutateurs NX-OS, ce qui permet à ce matériel de fonctionner avec Chef comme s'il s'agissait d'un hôte géré.
- Elle a également développé et maintenu un livre de cuisine Cisco Chef pour l'infrastructure NX-OS, disponible sur Chef Supermarket.
- Un dépôt public GitHub de livre de cuisine et de code de recette est également maintenu pour permettre le contrôle d'une large gamme de produits Cisco.
- L'infrastructure Cisco UCS est facilement gérée avec Chef grâce à un livre de recettes permettant l'intégration avec les contrôleurs de gestion intégrés.

Exemple de Chef — Installer et utiliser Chef

Cet exemple décrit comment installer Chef et l'utiliser pour installer Apache 2 sur un périphérique.

Installation du poste de travail du chef

Chef Workstation fournit un environnement d'exploitation complet. L'exemple suivant suppose que vous installez sur une machine virtuelle Ubuntu 18.04 LTS.

- Si votre machine est configurée avec un bureau standard, accédez à la page de téléchargement de Chef Workstation, recherchez le téléchargement pour Ubuntu 18.04 et installez-le automatiquement avec le gestionnaire de paquets Debian.
- Vous pouvez également installer à partir de la ligne de commande en copiant l'URL du package `.deb`.

```
wget https://packages.chef.io/files/stable/chef-workstation/0.12.20/ubuntu/18.04/chef-workstation_0.12.20-1_amd64.deb
sudo dpkg -i chef-workstation_0.12.20-1_amd64.deb
```

Exemple de Chef — Installer et utiliser Chef (suite)

Gestion de la configuration de base

- Après l'installation de Workstation, commencez à effectuer des changements de configuration sur les hôtes accessibles.
- Vous utiliserez la commande `chef-run` pour cela. La première fois que vous utilisez `chef-run`, vous pouvez être invité à accepter les conditions de licence pour l'utilitaire.
- Pour le premier exercice de configuration, vous fournirez les informations dont Chef a besoin pour installer le paquet `ntp`. Dans le processus, vous allez fournir le nom d'utilisateur distant, leur mot de passe sudo, le nom de la `cible` hôte distant et le nom du verbe de ressource.

```
chef-run -U myname -sudo <password> target package ntp action=install
```

- Lorsque le client est installé, la tâche lui est remise et le processus se termine. Vous revenez comme suit :

```
[✓] Packaging cookbook... done!  
[✓] Generating local policyfile... exporting... done!  
[✓] Applying package[ntp] from resource to target.  
└─ [✓] [target] Successfully converged package[ntp].
```

Exemple de Chef — Installer et utiliser Chef (suite)

Installer Chef Infra Client

- Chef Infra Client s'exécute localement sur des nœuds de calcul conventionnels.
- Chef Workstation peut amorçage Infra Client sur les nœuds cibles. Vous pouvez également préinstaller Infra Client sur des nœuds, par exemple, lors de la création de nouveaux nœuds sur un cloud public. Voici un exemple de script que vous pouvez exécuter sur un hôte cible pour ce faire.
- Le script utilise un programme d'installation fourni par le CHEF appelé Omnitruck pour ce faire. Une version Windows de ce script est également disponible qui s'exécute sur PowerShell :

```
#!/bin/bash
apt-get update
apt-get install curl
curl -L https://omnitruck.chef.io/install.sh | bash -s once -c current -p chef
```

- Notez que les paramètres indiqués ci-dessus installeront la dernière version du client Chef et n'épinglez pas la version.

Exemple de Chef — Installer et utiliser Chef (suite)

Prérequis pour le serveur Infra

- Avant d'installer Chef Infra Server, installez openssh-server et activez l'accès par clé. Vous devez également installer NTP pour la synchronisation temporelle. Vous pouvez le faire avec Chef, ou manuellement :

```
sudo apt-get install ntp ntpdate net-tools
```

- Sur un système Ubuntu, désactivez le service de synchronisation `timedatectl` par défaut pour éviter qu'il n'interfère avec la synchronisation NTP :

```
sudo timedatectl set-ntp 0
```

- Une fois NTP installé, assurez-vous qu'il est synchronisé avec un serveur de temps dans son pool par défaut. Cela peut prendre quelques minutes, alors répétez la commande jusqu'à ce que vous voyez ce qui suit :
- Lorsque cela apparaît, vous pouvez installer Chef Infra Server.

```
ntpstat  
synchronised to NTP server (198.60.22.240) at stratum 2  
time correct to within 108 ms  
polling server every 256 s
```

Exemple de Chef — Installer et utiliser Chef (suite)

Installer Chef Infra Server

- Chef Infra Server stocke la configuration et la fournit automatiquement aux Clients, lorsqu'ils sont interrogés, ce qui permet aux Clients de converger eux-mêmes vers l'état souhaité.
- Pour installer Chef Infra Server sur Ubuntu 18.04, vous pouvez effectuer des étapes similaires à l'installation manuelle de Workstation après avoir obtenu l'URL du package .deb. Au moment de l'écriture, la version stable actuelle était 13.1.13-1.

```
wget https://packages.chef.io/files/stable/chef-server/13.1.13/ubuntu/18.04/chef-server-core_13.1.13-1_amd64.deb  
sudo dpkg -i chef-server-core_13.1.13-1_amd64.deb
```

- Après l'installation de Chef Infra Server, exécutez la commande suivante pour lui dire de lire sa configuration par défaut, d'initialiser et de démarrer tous les services.

```
sudo chef-server-ctl reconfigure
```

Exemple de Chef — Installer et utiliser Chef (suite)

Installer Chef-Manage

- Vous pouvez également installer l'interface Web pour le serveur Chef. Cela peut être fait en entrant :

```
sudo chef-server-ctl install chef-manage
```

- Une fois le processus terminé, redémarrez le serveur et gérez les composants. Ce sont des opérations de chef, et peuvent prendre un certain temps, comme avant.

```
sudo chef-server-ctl reconfigure  
(lots of output)  
sudo chef-manage-ctl reconfigure --accept-license  
(lots of output)
```

- L'argument `--accept-license` empêche `chef-manage-ctl` de s'arrêter pour vous interroger sur les licences uniques de ce produit. Lorsque ce processus est terminé, vous pouvez visiter la console dans un navigateur via `https://<IP_OF_CHEF_SERVER>`.

Exemple de Chef — Installer et utiliser Chef (suite)

Terminer la configuration de la station de travail

- Avant que Chef Workstation puisse parler à votre serveur Infra, vous devez effectuer une petite configuration.
- Pour commencer, récupérez les clés générées lors de la configuration du serveur et stockez-les dans le dossier `/home/myname/.chef` créé lors de l'installation de Workstation :

```
cd /home/myname/.chef
scp myname@chefserver:./path/*.pem .
```

- `/path/` est le chemin de votre répertoire personnel sur le serveur vers le répertoire dans lequel le serveur a stocké les clés.
- Si vous n'utilisez pas l'authentification par clé de votre serveur, `scp` vous demandera votre mot de passe.
 - Le `.` après `user@host :` fait référence au répertoire personnel de votre utilisateur d'origine, à partir duquel le chemin d'accès est calculé.
 - L'expression générique trouve les fichiers se terminant par `.pem` à ce chemin. Le point de fermeture signifie copier dans le répertoire de travail courant.
 - Exécutez la commande `ls` à partir du dossier `.chef` pour voir si vos clés l'ont fait.

Exemple de Chef — Préparer l'utilisation d'un couteau

Préparez-vous à utiliser le couteau

Knife est un outil pour gérer des livres de cuisine, des recettes, des nœuds et d'autres ressources, et pour interagir avec le serveur Chef Infra.

- Dans le dossier `.chef`, modifiez le fichier (initialement vide) nommé `config.rb` et incluez les lignes de code suivantes, en les adaptant à votre environnement :

```
chef_server_url 'https://chefserver/organizations/<short_name>'
client_key '/home/<myname>/.chef/<userkey.pem>'
cookbook_path [
  '/home/<myname>/cookbooks'
]
data_bag_encrypt_version 2
node_name '<username>'
validation_client_name '<short_name>-validator'
validation_key '/home/<myname>/.chef/<short_name>-validator.pem'
```

Exemple de Chef — Préparer l'utilisation d'un couteau (suite)

- Enregistrez le fichier `config.rb`, puis créez le répertoire `/home/myname/cookbooks`.
- Enfin, émettez la commande `knife ssl fetch`.
- Si vous avez correctement configuré le `config.rb`, Knife consulte votre serveur, récupère son certificat et le stocke dans le répertoire.
- Chef le trouvera automatiquement lorsqu'il est temps de se connecter au serveur, fournissant l'assurance que le serveur est authentique.

Exemple de Chef — Préparer l'utilisation d'un couteau (suite)

Bootstrap un nœud cible avec knife

- Une fois que Knife est configuré, vous pouvez amorçage votre nœud cible.
- Pour amorçage, exécutez la commande suivante, en remplaçant les champs variables par vos informations. La commande est configurée pour utiliser l'authentification par clé à la machine cible.
- Les commandes redondantes `--sudo` et `--use-sudo-password` indiquent à Knife d'utiliser sudo pour terminer son travail.
- L'option `-P` fournit votre mot de passe `sudo` sur la machine cible.

`<name_for_your_node>` est un nom arbitraire. L'indicateur `--ssh-verify-host-key` et l'argument font que la commande ne s'arrête pas et ne demande pas votre permission de manière interactive si elle constate que vous ne vous êtes jamais connecté à ce serveur auparavant.

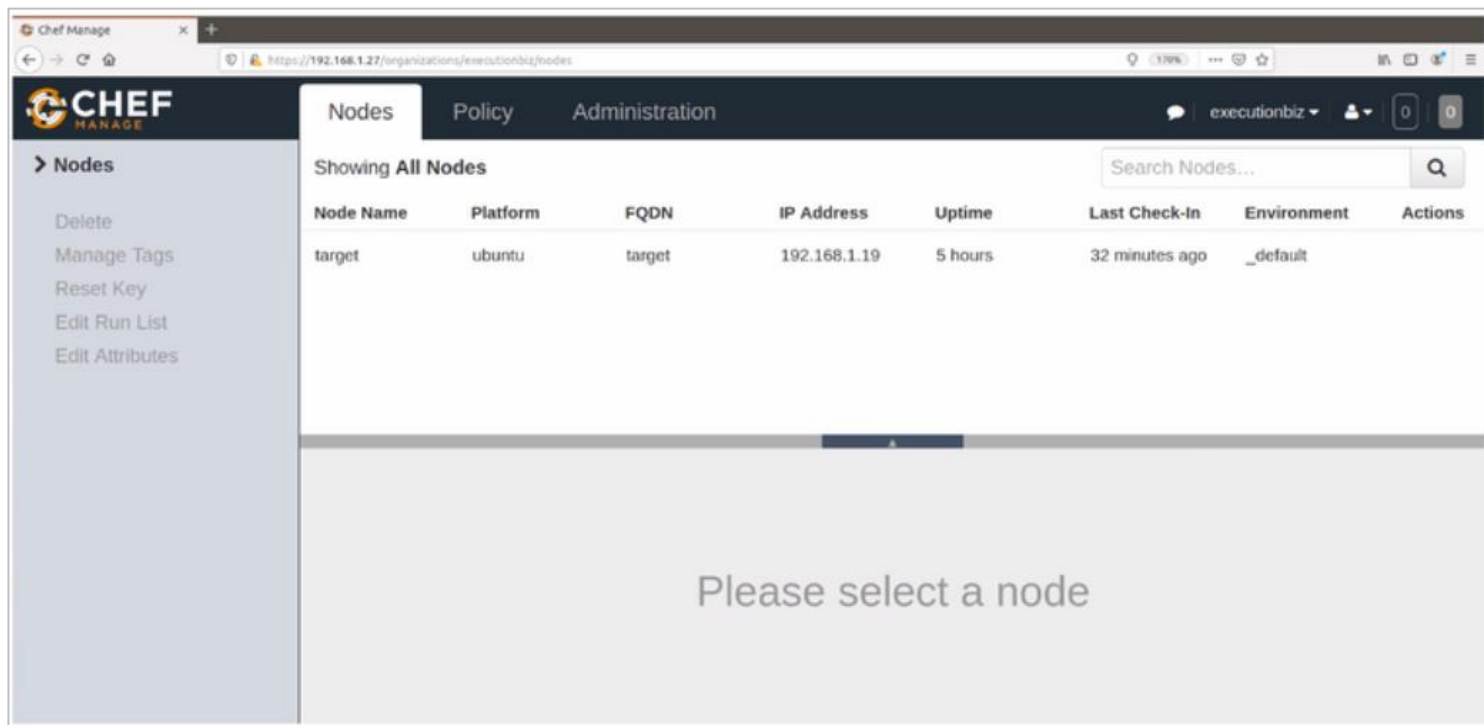
Exemple de Chef — Préparer l'utilisation d'un couteau (suite)

Si la commande fonctionne correctement, vous obtiendrez la sortie donnée. Notez que Chef a détecté l'installation précédente et ne l'a pas écrasée.

```
[target] [sudo] password for myname:
[target] -----> Existing Chef Infra Client installation detected
[target] Starting the first Chef Infra Client Client run...
[target] +-----+
✓ 2 product licenses accepted.
+-----+
[target] Starting Chef Infra Client, version 15.5.17
[target]
[target] Creating a new client identity for target using the validator key.
[target]
[target] resolving cookbooks for run list: []
[target]
[target] Synchronizing Cookbooks:
[target]
[target] Installing Cookbook Gems:
[target]
[target] Compiling Cookbooks...
[target]
[target] [2019-12-10T15:16:56-05:00] WARN: Node target has an empty run list.
[target] Converging 0 resources
[target]
[target]
[target] Running handlers:
[target]
[target] Running handlers complete
[target] Chef Infra Client finished, 0/0 resources updated in 02 seconds
[target]
```

Exemple de Chef—Préparer l'utilisation d'un couteau (suite)

Chef Manage affiche votre nœud cible : maintenant, si vous vérifiez à nouveau dans votre navigateur et actualisez Chef Manage, vous devriez voir que votre machine cible est désormais gérée par le serveur.



The screenshot shows the Chef Manage web interface in a browser. The URL is `https://192.168.1.27/organizations/executionbiz/nodes`. The interface has a dark header with the Chef logo and navigation tabs for 'Nodes', 'Policy', and 'Administration'. A left sidebar lists actions: 'Delete', 'Manage Tags', 'Reset Key', 'Edit Run List', and 'Edit Attributes'. The main content area, titled 'Showing All Nodes', contains a table with one node and a large grey area below it with the text 'Please select a node'.

Node Name	Platform	FQDN	IP Address	Uptime	Last Check-In	Environment	Actions
target	ubuntu	target	192.168.1.19	5 hours	32 minutes ago	_default	

Exemple de Chef – Tout mettre ensemble

Maintenant, vous allez tout utiliser ensemble pour créer une recette réelle, la pousser vers le serveur et dire au client de la machine cible de réquisitionner et de converger vers la nouvelle configuration.

- Pour commencer, créez un livre de cuisine pour construire un site Web simple. Accédez à votre répertoire de livres de cuisine, créez un nouveau livre de recettes appelé `apache2` et naviguez dans celui-ci.
- Vérifiez la structure du dossier du livre de recettes. Il existe des dossiers déjà préparés pour les recettes et les attributs. Ajoutez un répertoire et un sous-répertoire facultatifs pour conserver les fichiers dont votre recette a besoin.
- Les fichiers dans le sous-répertoire `/default` d'un répertoire `/files` d'un livre de recettes peuvent être trouvés uniquement par nom de recette, aucun chemin n'est requis.

Exemple de Chef – Tout mettre ensemble (suite)

- Maintenant, créez une page d'accueil pour votre site Web.

```
vi index.html
<html>
<head>
<title>Hello!</title>
</head>
<body>
<h1>HELLO!</h1>
</body>
</html>
```

- Sauvegardez le fichier et quittez. Accédez au répertoire des recettes, où Chef a déjà créé un fichier `default.rb` pour nous. Le fichier `default.rb` sera exécuté par défaut lorsque la recette est exécutée avec cette commande.

```
cd ../../recipes
```

- Ajoutez des données au fichier `default.rb` et modifiez à nouveau le fichier.

```
vi default.rb
```

Exemple de Chef – Tout mettre ensemble (suite)

- L'en-tête en haut est créé pour vous. En dessous, la recette effectue trois actions.
 - La première ressource que vous appelez `apt_update` gère le gestionnaire de paquets apt sur Debian.
 - La fonction de package est utilisée pour installer le paquet apache2 à partir de référentiels publics.
 - Enfin, vous utilisez la ressource `cookbook_file` pour copier le fichier `index.html` à partir de `/files/default` dans un répertoire sur le serveur cible.

```
#
# Cookbook:: apache2
# Recipe:: default
#
# Copyright:: 2019, The Authors, All Rights Reserved.
apt_update do
  action :update
end
package 'apache2' do
  action :install
end
cookbook_file "/var/www/html/index.html" do
  source "index.html"
  mode "0644"
end
```

Exemple de Chef – Tout mettre ensemble (suite)

- Enregistrez le fichier `.rb` par défaut, puis téléchargez le livre de cuisine sur le serveur.
- Vous pouvez ensuite confirmer que le serveur gère votre nœud cible.
- L'application Knife peut interagir avec votre éditeur préféré. Pour activer cette option, effectuez l'exportation suivante avec le nom de votre éditeur :

```
export EDITOR=vi
```

- Cela permet à la commande suivante de s'exécuter de manière interactive, en plaçant la définition du nœud dans vi pour vous permettre de la modifier manuellement.
- Comme vous pouvez le voir, l'expression `"recipe[apache2]"` a été ajoutée au tableau `run_list` qui contient une liste ordonnée des recettes que vous souhaitez appliquer à ce nœud.
- Enregistrez le fichier de la manière habituelle. Knife pousse immédiatement la modification vers le serveur Infra.

```
knife node edit target{
  "name": "target",
  "chef_environment": "_default",
  "normal": {
    "tags": [
    ]
  },
  "policy_name": null,
  "policy_group": null,
  "run_list": [
    "recipe[apache2]"
  ]
}
```

Exemple de Chef – Tout mettre ensemble (suite)

- Enfin, vous pouvez utiliser la commande `knife ssh` pour identifier le nœud, vous y connecter de manière non interactive en utilisant SSH et exécuter l'application `chef-client`.
- Si tout se passe bien, Knife vous renvoie un journal très long qui vous montre exactement le contenu du fichier qui a été écrasé, et confirme chaque étape de la recette au fur et à mesure qu'elle s'exécute.
- À ce stade, vous devriez être en mesure de pointer un navigateur vers l'adresse IP de la machine cible et de voir votre nouvelle page d'index.

Résumé des outils d'automatisation des outils d'automatisation

Résumé

Il s'agit d'une introduction de haut niveau à trois trousseaux d'outils DevOps modernes. Vous devriez maintenant être prêt à :

- Déployez et intégrez des versions gratuites des principaux composants d'Ansible, Puppet et/ou Chef sur une gamme de supports, des machines virtuelles de bureau aux machines virtuelles basées sur le cloud sur Azure, AWS ou d'autres plates-formes IaaS.
- Découvrez le langage déclaratif, le style de construction et d'organisation de chaque plate-forme sous forme de code, et obtenez une idée de la portée de sa bibliothèque de ressources, de plugins et d'intégrations.
- Apprenez à automatiser certaines des tâches informatiques courantes que vous pouvez effectuer au travail ou à résoudre les problèmes de déploiement et de gestion du cycle de vie que vous définissez vous-même, dans votre laboratoire à domicile.
- Des exercices pratiques et des travaux vous donneront une idée complète de la façon dont chaque plate-forme aborde les thèmes de configuration et vous aideront à surmonter les défis informatiques quotidiens.



1.4. Infrastructure en tant que code (IaaSC)

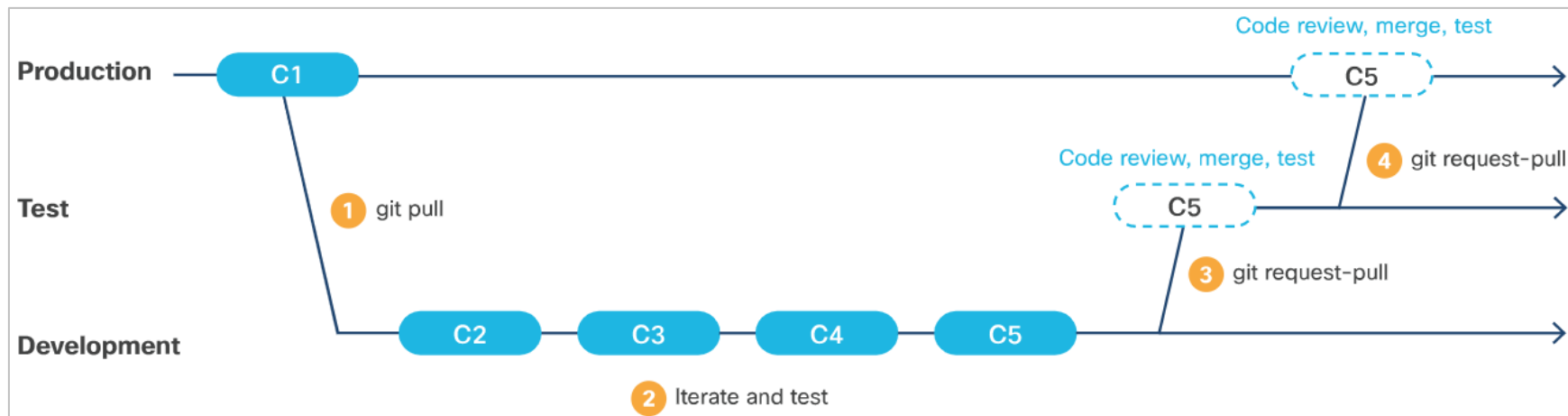
Infrastructure en tant que code

Pourquoi stocker l'infrastructure en tant que code ?

Le terme immuabilité fait référence à la maintenance des systèmes entièrement sous forme de code, n'effectuant aucune opération manuelle sur eux.

GITOPS : infrastructure moderne en tant que code

- GitOps est également appelé « opérations par demande de retrait. »
- Dans une configuration GitOps typique, vous pouvez maintenir un référentiel, tel qu'un dépôt privé sur GitHub, avec plusieurs branches appelées Développement, Testing/UAT et Production.



Pourquoi stocker l'infrastructure comme code ? (suite)

Où les GITOP peuvent-ils vous emmener ?

Lorsque toutes les procédures GITOPS, flux de travail et autres composants sont en place, les développeurs peuvent envisager la mise en œuvre de stratégies de déploiement d'élite.

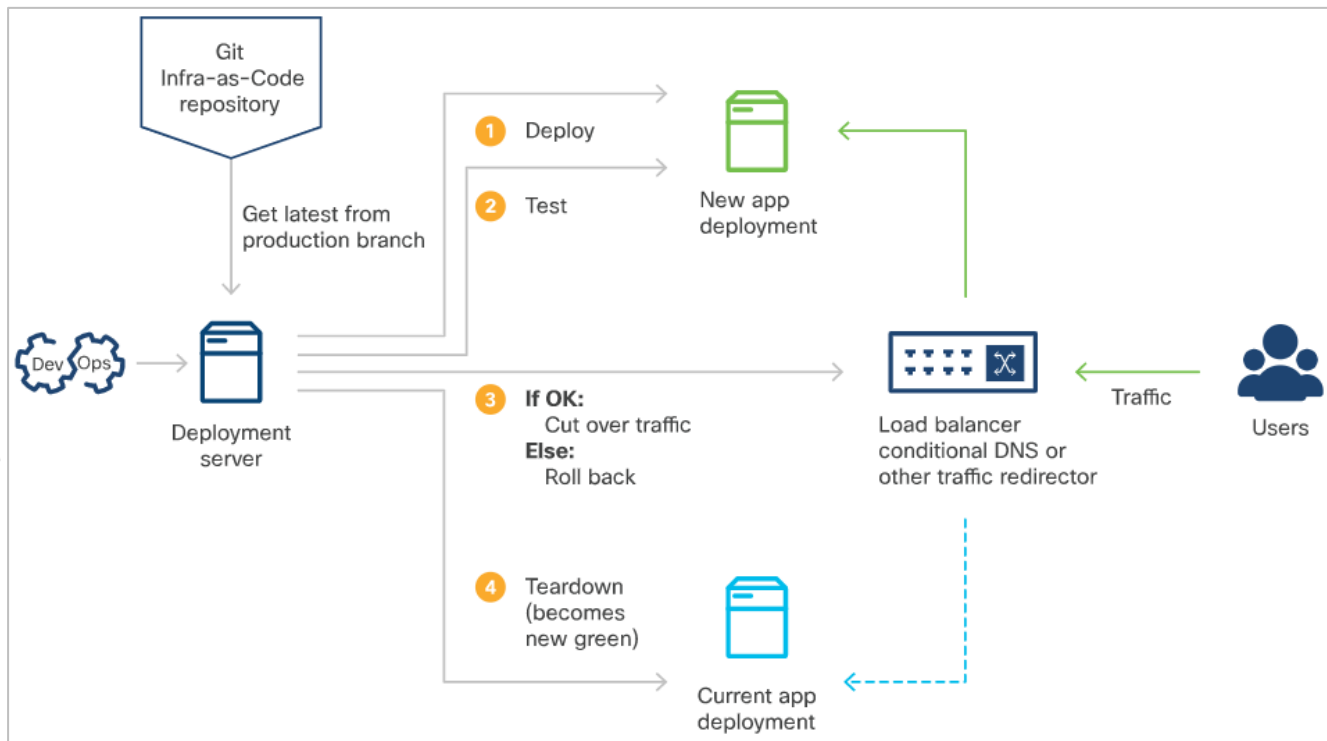
Déploiement bleu/vert

- Le déploiement bleu/vert est une méthode permettant de réduire ou d'éliminer les temps d'arrêt dans les environnements de production.
- Il est nécessaire de maintenir deux environnements de production identiques (pas nécessairement, bleu et vert. Deux couleurs telles que le rouge et le noir feront l'affaire).
- Il est également nécessaire de développer la capacité de rediriger rapidement le trafic d'applications vers l'un ou l'autre.

Remarque : Certains praticiens de DevOps font la différence entre les stratégies bleu/vert et rouge/noir. Ils disent qu'en bleu/vert, le trafic est progressivement migré d'un environnement à l'autre, de sorte qu'il touche les deux systèmes pendant un certain temps ; alors qu'en rouge/noir, le trafic est coupé à la fois.

Pourquoi stocker l'infrastructure comme code ? (suite)

- Une version est déployée dans l'environnement qui n'est pas actuellement utilisé (Vert). Après le test d'acceptation, redirigez le trafic vers cet environnement.
- Si des problèmes sont rencontrés, retournez le trafic vers l'environnement d'origine (bleu).
- Si le déploiement Green est jugé adéquat, les ressources appartenant au déploiement Blue peuvent être abandonnées et les rôles échangés pour la prochaine version.



Pourquoi stocker l'infrastructure comme code ? (suite)

Test des canaris

- Le test Canari est similaire au déploiement roulant bleu/vert, mais un peu plus délicat.
- La migration entre les anciens et les nouveaux déploiements est effectuée client par client (voire utilisateur par utilisateur).
- La migration vise à réduire les risques et à améliorer la qualité de la rétroaction.



1.5. Automatisierung des Tests

Test et validation automatisés

- En utilisant des outils de test unitaire tels que pytest, les développeurs peuvent construire un environnement où le code peut être testé automatiquement lorsque des modifications sont apportées.
- Les frameworks de tests unitaires font des tests une partie de la base de code, en suivant le code à travers les validations de développeurs, les requêtes d'extraction et les portes de révision de code vers QA/Test et Production. Ce cadre de test unitaire est utile dans les environnements de développement pilotés par des tests (TDD).

Les défis du test d'un réseau

- Le comportement et les performances d'un réseau réel sont collectifs, maintenus par les configurations de nombreux équipements et logiciels discrets.
- Dans les environnements traditionnels, la connectivité et la fonctionnalité sont maintenues manuellement sur de nombreux équipements individuels via diverses interfaces. C'est difficile, chronophage, extrêmement sujette aux erreurs et risqué.
- À mesure que les réseaux deviennent plus complexes et transportent un trafic plus diversifié et sensible aux performances, les risques pour la sécurité et la dégradation des performances sont plus élevés.

Test et validation automatisés (suite)

Tester les réseaux définis par logiciel (SDN)

Cisco a réalisé d'énormes progrès dans le développement de réseaux définis par logiciel (SDN) et de middleware qui permettent aux ingénieurs d'aborder un réseau physique en tant qu'entité programmable unique. Dans le cas de Cisco, cela inclut :

- Infrastructure centrée sur les applications (ACI)
- Centre d'architecture de réseau numérique (Cisco DNA Center)
- L'API REST et les kits SDK permettent l'intégration avec des outils d'automatisation comme Ansible, Puppet et Chef

Système de test automatisé Python (PyATS)

- Python Automated Test System (PyATS) est une solution de test et de validation de périphériques réseau basée sur Python.
- PYATS est né comme base Python de bas niveau pour le système de test dans son ensemble.
- Son système de bibliothèque de niveau supérieur, Génie, fournit les API et les bibliothèques nécessaires qui pilotent et interagissent avec les périphériques réseau, et effectuent les tests réels.

Test et validation automatisés (suite)

PYATS a plusieurs caractéristiques clés :

- Le framework et les bibliothèques PyATS peuvent être exploités dans n'importe quel code Python.
- Il est modulaire et comprend des composants tels que AETest et Easy.py.
- Une CLI permet d'interroger rapidement les réseaux en direct, d'extraire les faits et d'automatiser l'exécution des scripts de test et d'autres analyses médico-légales.
- PYATS fournit une énorme bibliothèque d'interfaces à Cisco et à d'autres infrastructures via une gamme d'interfaces.
- PyATS peut consommer, analyser et implémenter des topologies décrites dans JSON, en tant que modèles YANG et à partir d'autres sources.
- PYATS peut également être intégré avec des outils d'automatisation pour la construction, le provisionnement et le démontage.

Exemple du système pyATS

Le contenu suivant montre comment utiliser PyATS pour créer et appliquer des tests.

Environnements virtuels

L'outil PyATS est mieux installé pour un travail personnel dans un environnement virtuel Python (venv).

- Un venv est un environnement copié à partir de votre environnement de base, mais gardé séparé de celui-ci.
- Cela vous permet d'éviter d'installer des logiciels susceptibles de modifier définitivement l'état de votre système.
- Des environnements virtuels existent dans les dossiers de votre système de fichiers. Lorsqu'ils sont créés, ils peuvent être activés, configurés à volonté et les composants qui y sont installés peuvent être mis à jour ou modifiés sans que les modifications soient reflétées dans la configuration de votre hôte.
- La possibilité de créer des environnements virtuels est native à Python 3, mais Ubuntu 18.04 peut nécessiter l'installation d'un paquet python3-venv séparément.

Exemple pyATS (suite)

Pour créer un venv sur Ubuntu 18.04 :

- Assurez-vous que `python3-pip`, le gestionnaire de paquets Python3, est en place et installez `git`.
- Créez un nouvel environnement virtuel dans le répertoire de votre choix.
- Venv crée le répertoire de travail et la structure de dossier spécifiés contenant des fonctions et des artefacts décrivant la configuration de cet environnement. À ce stade, vous pouvez cd au `monprojet` et activer le venv.

Installation de PYATS

- Installez les PYATS à partir du référentiel public de paquets Pip (PyPI).
- Vérifiez qu'il a été installé en listant l'aide, à l'aide de `pyats --help`.
- Cloner le repo des exemples de scripts PyATS, géré par Cisco DevNet, qui contient des exemples de fichiers.

Remarque : Vous pouvez voir des erreurs « Échec de la construction de la roue pour...<wheelname> » lors de l'installation de PYATS. Vous pouvez ignorer ces erreurs en toute sécurité car `pip` a un plan de sauvegarde pour ces échecs et les dépendances sont installées malgré les erreurs signalées.

Exemple pyATS (suite)

Syntaxe du cas de test PyATS

- La syntaxe de déclaration de test pour PyATS est inspirée par les frameworks de test d'unité Python comme `pytest`.
- Il prend en charge les instructions de test de base, telles que l'affirmation qu'une variable a une valeur donnée, et ajoute à cela la possibilité de fournir explicitement des résultats.

Scripts et jobs PyATS

- Un script PyATS est un fichier Python où les tests PyATS sont déclarés.

Fichier testbed PyATS

- Un testbed peut être un seul fichier YAML ou peut être assemblé par programme à partir de YAML et Python.
- Le fichier de testbed est une entrée essentielle pour le reste de la bibliothèque et de l'écosystème PyATS car il fournit des informations à l'infrastructure pour charger le bon ensemble d'API de bibliothèque pour chaque périphérique, et comment communiquer efficacement avec eux.
- Les fichiers de testbed réels pour les topologies volumineuses peuvent être longs, profondément imbriqués et complexes.

Exemple pyATS (suite)

Bibliothèque PyATS : Génie

- Génie est le système de bibliothèque de niveau supérieur PyATS qui fournit des API pour interagir avec les périphériques et une interface de ligne de commande puissante pour la topologie et la gestion et l'interrogation des périphériques.
- Lorsqu'il est installé, il ajoute ses fonctionnalités et fonctionnalités dans le framework PyATS.

Fin du Chapitre 1

